

Java2 Day02 핵심 정리

람다식, 함수형 인터페이스, forEach, Comparator.comparing, 표준 함수형 인터페이스 정리

주제: 람다식과 함수형 인터페이스

대상 독자: 인터페이스를 배운 뒤 람다식을 처음 학습하는 초보자

작성 기준 자료: Day2 TXT, Day2 원본 PDF, java2prjDay2 day2 소스 코드

생성일: 2026-05-21

람다식

메서드 구현을 짧은 식으로 표현하는 문법입니다.

함수형 인터페이스

추상 메서드가 하나뿐인 인터페이스입니다.

forEach

리스트의 각 요소마다 같은 동작을 실행합니다.

표준 함수형 인터페이스

Runnable, Consumer, Supplier, Predicate, Function을 배웁니다.

0. 목차와 학습 로드맵

단원	학습 내용
1	Day02 전체 개요
2	함수형 인터페이스 직접 만들기
3	람다식 문법 줄이기
4	forEach와 Consumer
5	Comparator.comparing으로 객체 정렬
6	표준 함수형 인터페이스 5개
7~9	실습 방향, 연습 문제, 최종 요약

1. 먼저 람다식이 함수형 인터페이스 구현이라는 점을 이해합니다.
2. 이름 있는 클래스, 익명 클래스, 람다식을 비교합니다.
3. 표준 함수형 인터페이스의 입력과 반환 형태를 외웁니다.
4. forEach와 정렬 예제로 람다식을 직접 읽어 봅니다.
5. 연습 문제로 Runnable, Consumer, Supplier, Predicate, Function을 구분합니다.

원본 자료	확인 내용	반영 방식
2일차.txt	전체 내용 확인	학습 목표, 오전/오후 실습, 표준 함수형 인터페이스 반영
Day2 원본 PDF 4개	파일 존재, PDF 헤더, 페이지 구조 확인	람다식/함수형 인터페이스 주제 반영
workspace/java2prjDay2/src/day2	50개 파일 구조와 대표 예제 확인	HelloInterface, Moving, Runnable/Consumer/Supplier 등 코드 반영

확인 제한: PDF 본문 세부 텍스트는 완전 추출하지 못했습니다. TXT 전체 내용과 실제 소스 코드 확인 내용을 중심으로 재구성했습니다.

1. Day02에서 배우는 Java 문법

Day02의 중심은 **람다식**입니다. 람다식은 메서드를 더 짧게 쓰는 문법처럼 보이지만, Java에서는 메서드가 혼자 존재할 수 없기 때문에 실제로는 **함수형 인터페이스의 추상 메서드를 구현하는 코드**입니다.

개념	뜻	예
함수형 인터페이스	구현해야 할 추상 메서드가 하나뿐인 인터페이스	Runnable, Consumer<T>
람다식	함수형 인터페이스 구현을 짧게 적는 문법	() -> System.out.println("hi")
익명 클래스	이름 없이 즉석에서 만드는 구현 클래스	new Runnable() { ... }
forEach	각 요소마다 Consumer를 실행	list.forEach(x -> ...)
Comparator.comparing	정렬 기준을 람다식으로 쉽게 작성	Comparator.comparing(s -> s.getName())

핵심: 람다식은 아무 곳에서나 쓰는 문법이 아닙니다. **추상 메서드가 하나뿐인 인터페이스**가 있어야 사용할 수 있습니다.

2. 함수형 인터페이스 직접 만들기

수업 예제에서는 HelloInterface를 만들고, 같은 기능을 세 가지 방식으로 구현합니다.

```
@FunctionalInterface
public interface HelloInterface {
    void print();
}
```

@FunctionalInterface는 이 인터페이스가 함수형 인터페이스인지 컴파일러가 검사하게 해 줍니다. 추상 메서드가 두 개 이상이면 오류가 납니다.

2-1. 이름 있는 클래스로 구현

```
class HelloImp implements HelloInterface {
    @Override
    public void print() {
        System.out.println("hello");
    }
}

HelloInterface h = new HelloImp();
h.print();
```

가장 기본적인 방식입니다. 클래스 이름이 있고, 여러 번 재사용할 수 있습니다.

2-2. 익명 클래스로 구현

```
HelloInterface h = new HelloInterface() {
    @Override
    public void print() {
        System.out.println("익명 클래스로 구현");
    }
};

h.print();
```

한 번만 사용할 구현이라면 굳이 새 파일이나 새 클래스 이름을 만들지 않아도 됩니다. 이때 쓰는 방식이 익명 클래스입니다.

2-3. 람다식으로 구현

```
HelloInterface h = () -> System.out.println("람다식으로 구현");
h.print();
```

람다식은 익명 클래스보다 훨씬 짧습니다. 단, HelloInterface처럼 구현할 추상 메서드가 하나뿐이어야 합니다.

3. 람다식 문법 줄이기

기존 메서드	람다식	설명
<code>int max(int a, int b) { return a > b ? a : b; }</code>	<code>(a, b) -> a > b ? a : b</code>	한 줄 반환이면 return 생략
<code>void print(String name) { System.out.println(name); }</code>	<code>name -> System.out.println(name)</code>	매개변수 1개면 괄호 생략 가능
<code>int square(int x) { return x * x; }</code>	<code>x -> x * x</code>	타입은 대부분 추론
<code>int getRandom() { return (int)(Math.random() * 6); }</code>	<code>() -> (int)(Math.random() * 6)</code>	매개변수가 없으면 () 필요

람다식에서 ->는 "왼쪽 입력을 받아서 오른쪽 코드를 실행한다" 정도로 이해하면 됩니다.

4. forEach와 Consumer

forEach는 리스트 안의 요소를 하나씩 꺼내서 같은 동작을 실행합니다. 내부적으로는 Consumer<T>를 받습니다.

```
ArrayList<String> list = new ArrayList<>();
list.add("장주원");
list.add("이미현");
list.add("김두식");

list.forEach(name -> System.out.println(name));
```

name은 리스트에서 하나씩 꺼낸 값입니다. 위 코드는 기존 for문보다 짧고 읽기 쉽습니다.

```
Consumer<String> c = str -> System.out.println("출력: " + str);
c.accept("람다식");
```

Consumer는 입력은 받지만 반환값은 없습니다. "소비한다"는 이름처럼 값을 받아 출력하거나 저장하거나 처리할 때 씁니다.

5. Comparator.comparing으로 객체 정렬

```
public class Moving {
    String name;
    String alias;
    int power;

    public int getPower() {
        return power;
    }

    public String getAlias() {
        return alias;
    }
}
```

```
Collections.sort(list, (o1, o2) -> o1.getPower() - o2.getPower());
Collections.sort(list, Comparator.comparing(m -> m.getAlias()));
```

첫 줄은 직접 비교하는 람다식입니다. 두 번째 줄은 Comparator.comparing을 사용해서 "별칭으로 정렬하라"고 더 읽기 쉽게 표현한 코드입니다.

정렬 기준만 알려주면 된다: 객체 정렬에서 중요한 것은 객체 전체가 아니라 이름, 별칭, 파워처럼 비교할 기준값입니다.

6. 표준 함수형 인터페이스 5개

람다식을 쓰려고 매번 인터페이스를 새로 만들 필요는 없습니다. Java가 자주 쓰는 함수형 인터페이스를 이미 제공합니다.

인터페이스	메서드	입력	반환	쉬운 설명
Runnable	run()	없음	없음	그냥 실행할 코드
Consumer<T>	accept(T t)	1개	없음	받은 값을 소비
Supplier<T>	get()	없음	1개	값을 공급
Predicate<T>	test(T t)	1개	boolean	조건 검사
Function<T,R>	apply(T t)	1개	1개	입력을 다른 결과로 변환

6-1. Runnable

```
Runnable r = () -> System.out.println("실행할 코드");
r.run();
```

6-2. Supplier

```
Supplier<Integer> random = () -> (int)(Math.random() * 5);
int value = random.get();
```

6-3. Predicate

```
Predicate<Integer> even = n -> n % 2 == 0;
boolean result = even.test(10);
```

6-4. Function

```
Function<Integer, Integer> doubleValue = n -> n * 2;
int result = doubleValue.apply(10);
```

7. 오전/오후 실습 방향

- 함수형 인터페이스를 직접 만들고 이름 있는 클래스, 익명 클래스, 람다식으로 각각 구현해 보기
- Moving 객체를 파워순으로 정렬해 보기
- ArrayList<String>을 forEach로 출력해 보기
- Runnable, Consumer, Supplier, Predicate, Function을 각각 람다식으로 구현해 보기

8. 연습 문제와 정답 해설

문제

1. 람다식을 사용하려면 어떤 인터페이스가 필요할까?
2. Consumer<T>와 Supplier<T>의 차이는 무엇인가?
3. Predicate<T>의 반환 타입은 무엇인가?
4. Comparator.comparing(s -> s.getName())은 어떤 기준으로 정렬하는가?

정답 및 해설

번호	정답 해설
1	추상 메서드가 하나뿐인 함수형 인터페이스가 필요합니다.
2	Consumer는 값을 받아 처리하고 반환은 없으며, Supplier는 입력 없이 값을 반환합니다.
3	Predicate는 조건 검사 결과인 boolean을 반환합니다.
4	객체의 getName() 반환값, 즉 이름을 기준으로 정렬합니다.

9. 원본 출처와 확인 불가 항목

항목	이유	필요한 추가 자료
Day2 원본 PDF 세부 본문 전체 추출	PDF 텍스트 추출 도구 없음	텍스트 추출본 또는 OCR 결과
코드 실행 결과	이번 자료 제작에서는 실행 검증하지 않음	Eclipse 또는 javac 실행 환경 확인

10. 마지막 핵심

주제	반드시 기억할 말
람다식	함수형 인터페이스의 추상 메서드를 짧게 구현하는 문법 이다.
함수형 인터페이스	추상 메서드가 하나뿐인 인터페이스다.
익명 클래스	이름 없이 즉석에서 만드는 구현 클래스다.
forEach	각 요소마다 같은 작업을 실행한다.
표준 함수형 인터페이스	입력과 반환 형태에 따라 골라 쓰면 된다.